

DevTools Companion for Angular

User Manual

Version 1.1 — June 2026

Firefox & Chrome Browser Extension

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 What You Can Do	4
1.2 Browser Support	4
2. Installation	5
2.1 Firefox — Permanent Installation (XPI)	5
2.2 Firefox — Temporary Installation (Developers)	5
2.3 Firefox Developer Edition / Nightly	5
2.4 Chrome Installation	5
3. Getting Started	7
3.1 Detection Badge	7
3.2 Production Mode Banner	7
4. Panel Overview	8
4.1 Tabs	8
5. Components Tab	9
5.1 Component Tree (Left Pane)	9
5.2 Toolbar Actions	9
5.3 Component Detail (Right Pane)	9
Component Info	10
@Input() Bindings	10
@Output() Events	11
Properties	11
5.4 Action Buttons	12
6. HTTP Tab	13
6.1 Request List (Left Pane)	13
6.2 Filters	13
6.3 Request Detail (Right Pane)	14
7. Store Tab	15
7.1 Action Log (Left Pane)	15
7.2 State Tree (Right Pane)	15
8. Router Tab	16
8.1 Current Route	16
8.2 Navigation History	16
8.3 Route Configuration Tree	16
9. DI Tab (Dependency Injection)	17

9.1 Service List (Left Pane)	17
9.2 Service Detail (Right Pane)	17
10. Performance Tab.....	18
10.1 Controls	18
10.2 Metrics Cards.....	18
10.3 CD Timeline	18
10.4 Most Checked Components.....	18
10.5 Subscription Leaks	19
11. Sources Tab	20
11.1 File Browser (Left Pane)	20
11.2 Source Viewer (Right Pane)	20
11.3 Jump to Source.....	20
12. Debug Tab	21
13. Settings	22
13.1 Theme	23
13.2 HTTP Monitor Settings	23
13.3 Performance Settings	23
14. Known Limitations	24
15. Troubleshooting.....	25
The DevTools Angular tab does not appear in DevTools	25
The badge shows "No Angular" on an Angular application.....	25
The component tree is empty in production mode	25
HTTP requests are not appearing in the monitor	25
Source maps not detected	25
The extension slows down after long sessions	26
16. Privacy & Data.....	27
Appendix — Keyboard Shortcuts	28

1. Introduction

DevTools Companion for Angular is a browser extension that adds a dedicated Angular debugging panel to your browser's built-in Developer Tools. It gives front-end developers complete visibility into any running Angular application — component state, HTTP traffic, store actions, routing events, change detection metrics, and TypeScript source files — without modifying the application or switching browsers.

1.1 What You Can Do

Area	Capability
Component Tree	Browse the full Angular component hierarchy, read <code>@Input()/@Output()</code> values, and edit inputs live.
HTTP Monitor	Capture every XHR and <code>fetch()</code> call with request/response headers, bodies, and timing.
State Management	Inspect NgRx, Akita, and NGXS stores; time-travel through dispatched actions.
Router	See the active route, navigation history, guard results, and the full route config tree.
DI Inspector	List services injected into any component and inspect their runtime state.
Performance	Record Angular change detection cycles and identify components that should use <code>OnPush</code> .
Source Maps	Browse and read the original TypeScript source files when <code>.map</code> files are served.

1.2 Browser Support

Browser	Minimum Version
Firefox (Stable / ESR)	115+
Firefox Developer Edition	Any current release
Firefox Nightly	Any current release
Google Chrome	120+

The extension is compatible with Angular applications from version 12 through 18+, in both development mode and production mode (some features are restricted in production builds — see Section 7).

2. Installation

2.1 Firefox — Permanent Installation (XPI)

This method installs the extension permanently so it survives browser restarts. It requires Firefox Developer Edition, Firefox Nightly, or Firefox ESR.

Step 1 — Enable unsigned extension installs

- Open Firefox and type `about:config` in the address bar.
- Accept the risk warning.
- Search for `xpinstall.signatures.required` and toggle the value to **false**.

Step 2 — Install the XPI file

- Open the Firefox menu (≡) → **Add-ons and Themes** (or type `about:addons`).
- Click the gear icon ⚙ → **Install Add-on From File...**
- Select the file `devtools-companion-angular.xpi`.
- Confirm the installation dialog.

Note: The `xpinstall.signatures.required` preference is only effective in Firefox Developer Edition, Nightly, and ESR. Standard Firefox stable releases cannot install unsigned extensions through this method.

2.2 Firefox — Temporary Installation (Developers)

Temporary installations do not require signature changes and are useful during development. The extension is removed when Firefox is closed.

- Open `about:debugging#/runtime/this-firefox` in Firefox.
- Click Load Temporary Add-on...
- Navigate to the `angular-inspector/` folder and select `manifest.json`.

The extension appears in the list with an active session indicator. Click **Reload** next to its name after any source code change.

2.3 Firefox Developer Edition / Nightly

These builds have signature verification disabled by default. Follow Steps 2 of Section 2.1 directly — no `about:config` change needed.

2.4 Chrome Installation

Load the extension as an unpacked extension during development, or distribute it via the Chrome Web Store.

Unpacked load (development):

- Open Chrome and navigate to `chrome://extensions`.
- Enable Developer mode (top-right toggle).
- Click Load unpacked.
- Select the `angular-inspector/` project folder.

Chrome Web Store:

Once the extension is published on the Chrome Web Store, users can install it with a single click from the store listing page.

3. Getting Started

After installation, follow these steps to open the extension panel:

- Open any Angular application in your browser (localhost or a remote URL).
- Open Developer Tools by pressing **F12** or **Ctrl+Shift+I** (**Cmd+Option+I** on macOS).
- Find the DevTools Angular tab in the DevTools tab bar. If the tab bar is crowded, it may be hidden behind the » overflow button.
- Click the DevTools Angular tab to open the panel.

3.1 Detection Badge

The badge in the top-right corner of the panel shows the Angular detection status:

Badge	Meaning
Angular (Ivy) — green	Angular application detected in development mode. All features are available.
Production Mode — amber	Angular detected in production mode. Component tree and DI inspector may be limited.
Detecting Angular... — grey	The extension is still scanning. Wait up to 10 seconds, then reload the page if it persists.
No Angular — red	No Angular application found on the current page.

3.2 Production Mode Banner

If the page uses a production Angular build, a yellow banner appears with an **Enable Debug Tools** button. Click it to invoke Angular's `enableDebugTools()` API, which re-enables component inspection. This works in most production builds, but some heavily optimised bundles may not support it.

Tip: If the badge shows "Detecting Angular..." for more than 10 seconds, reload the page with the DevTools panel already open. Some single-page applications complete their bootstrap after the content script first runs.

4. Panel Overview

The DevTools Angular panel is divided into three vertical regions:

Region	Description
Header (36 px)	Contains the extension logo, the tab bar, the Angular detection badge, and the refresh button.
Status Banner (optional)	Appears below the tab bar when a notable condition is detected (production mode, no Angular, settings saved).
Tab Panel (remaining)	The active tab's content. Most tabs use a resizable two-pane layout with a list on the left and a detail view on the right.

4.1 Tabs

Tab	Purpose
☐ Components	Component tree browser and live property inspector.
☐ HTTP	Real-time HTTP request monitor with filtering and detail drawer.
☐ Store	NgRx / Akita / NGXS store viewer with action log and time-travel.
☐ Router	Active route details, navigation history, and route config tree.
☐ DI	Dependency injection inspector for the selected component.
 Performance	Change detection profiler and subscription leak detector.
☐ Sources	TypeScript source file browser powered by source maps.
☐ Debug	Extension internals log for troubleshooting.
☐ Settings	Theme and behaviour configuration.

5. Components Tab

The Components tab is the main debugging panel. It displays the full Angular component hierarchy rendered on the current page.

5.1 Component Tree (Left Pane)

Element	Description
Bold text	Component class name (e.g. AppComponent, HeroListComponent).
Grey text	CSS selector used in the parent template (e.g. app-hero-list).
OnPush badge	Indicates the component uses the OnPush change detection strategy.
 /  arrow	Expand or collapse child components.

5.2 Toolbar Actions

Control	Function
Search field	Filter the tree in real time by component class name or selector.
 (Picker)	Activate element-pick mode. Click any element on the inspected page to jump to its component in the tree.
 (Refresh)	Force a new scan of the component tree (useful after dynamic DOM changes).

5.3 Component Detail (Right Pane)

Click any tree node to select it. A red overlay highlights the component's host element on the page. The right pane shows four sections:

The screenshot shows the DevTools Angular Components panel. On the left, a component tree is displayed with the following structure:

- AppComponent <wes-root>
 - _MatSidenavContainer <mat-sidenav-container>
 - _MatSidenav** <mat-sidenav> (highlighted in red)
 - _MatToolbar <mat-toolbar>
 - _MatNavList <mat-nav-list>
 - _MatSidenavContent <mat-sidenav-content>
 - _MatToolbar <mat-toolbar>

On the right, the 'COMPONENT' table is shown:

COMPONENT	
Class	_MatSidenav
Selector	mat-sidenav
CD Strategy	OnPush
Element	mat-sidenav
File	mat-sidenav.component.ts
@INPUT() BINDINGS	
fixedInViewport	false
fixedTopGap	0
fixedBottomGap	0
position	"start"
mode	"side"
disableClose	false

Component Info

Field	Content
Class	TypeScript class name.
Selector	CSS selector of the component.
CD Strategy	Default or OnPush.
Element	HTML tag of the host element.

COMPONENT

Class	_MatSidenav
Selector	mat-sidenav
CD Strategy	OnPush
Element	mat-sidenav
File	mat-sidenav.component.ts

@Input() Bindings

All @Input() properties with their current runtime values. Values are **editable**: click a value field, type a new value in valid JSON format, and press **Tab** or click away. The change is applied immediately on the page without a reload.

@INPUT() BINDINGS

fixedInViewport	false
fixedTopGap	0
fixedBottomGap	0
position	"start"
mode	"side"
disableClose	false
autoFocus	"dialog"
opened	true

@Output() Events

Lists all @Output() EventEmitter properties declared on the component.

@OUTPUT() EVENTS

openedChange	EventEmitter_ 1 sub async
opened #	AnonymousSubject no subs
openedStart	AnonymousSubject no subs
closed #	AnonymousSubject no subs
closedStart	AnonymousSubject no subs
positionChanged #	EventEmitter_ 1 sub

Properties

All public non-@Input() properties of the component with their current values.

PROPERTIES

Expand All

Collapse All

_elementRef	{ "nativeElement": "[mat-sidenav]" }
_focusTrapFactory	{ "_checker": { "_platform": { "_platformId": "browser", "isBrowser": true, "EDGE": false, ... } } } ▼ expand
_focusMonitor	{ "_ngZone": { "hasPendingMacrotasks": false, "hasPendingMicrotasks": false, "isStable": true, ... } } ▼ expand
_platform	{ "_platformId": "browser", "isBrowser": true, "EDGE": false, "TRIDENT": false, "BLINK": false, ... } ▼ expand
_ngZone	{ "hasPendingMacrotasks": false, "hasPendingMicrotasks": false, "isStable": true, "onUnstable": ... } ▼ expand
_interactivityChecker	{ "_platform": { "_platformId": "browser", "isBrowser": true, "EDGE": false, "TRIDENT": false, ... } } ▼ expand

5.4 Action Buttons

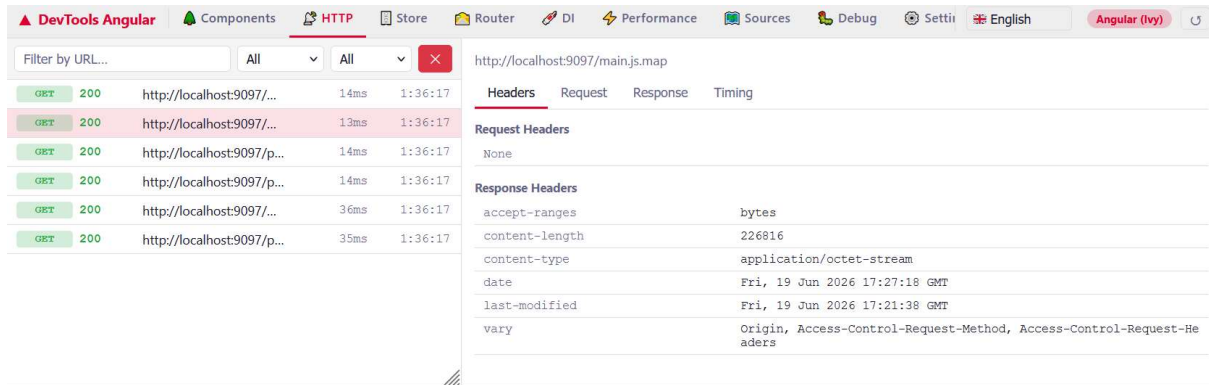
Button	Action
Copy JSON	Copies the full component state snapshot to the clipboard as a JSON string.
Clear Highlight	Removes the red overlay from the inspected page.

A rectangular button with rounded corners, a light gray background, and a thin border. The text "Copy JSON" is centered in a dark gray font.A rectangular button with rounded corners, a light gray background, and a thin border. The text "Clear Highlight" is centered in a dark gray font.

6. HTTP Tab

The HTTP tab captures all network requests made by the application through `XMLHttpRequest` and `fetch()`, including calls from Angular's `HttpClient`.

The HTTP interceptor is installed when the page loads. Requests made before the extension was active on a tab may not appear in the log.



6.1 Request List (Left Pane)

Column	Description
Method	Colour-coded badge: GET (green), POST (blue), PUT (amber), DELETE (red), PATCH (grey).
Status	HTTP status code: green for 2xx, amber for 3xx, red for 4xx/5xx.
URL	Full request URL (truncated in the cell; hover for the complete URL).
Duration	Time from request start to response received, in milliseconds.
Time	Local clock time when the request was sent.

A **red left border** on a row indicates the request exceeded the slow-request threshold (default: 1000 ms; configurable in Settings).

The **red numeric badge** on the HTTP tab counts 4xx/5xx responses since the log was last cleared.

6.2 Filters

Filter	Options
URL	Free text — filters by substring match against the full URL.
Method	All / GET / POST / PUT / DELETE / PATCH
Status	All / 2xx / 3xx / 4xx / 5xx
× Clear	Clears the entire request log.

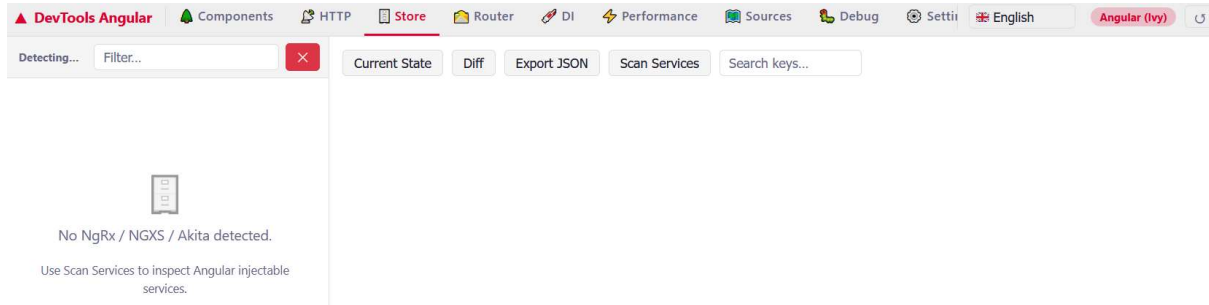
6.3 Request Detail (Right Pane)

Click any row to open the detail pane with four sub-tabs:

Sub-tab	Content
Headers	Request and response headers.
Request	Request body (JSON is auto-formatted).
Response	Response body (JSON is auto-formatted, up to 50 KB).
Timing	Start time, end time, total duration, and HTTP status code.

7. Store Tab

The Store tab inspects state management libraries used by the Angular application. **NgRx**, **Akita**, and **NGXS** are detected automatically. The badge in the toolbar shows the library name once detected.



7.1 Action Log (Left Pane)

Shows all dispatched actions in reverse chronological order (newest at the top). For each action:

- Action type name.
- Payload (collapsed; click to expand).
- Timestamp.

Click any action to **time-travel**: the state tree in the right pane rewinds to the snapshot at that moment.

Use the **search field** to filter actions by type name.

The **X button** clears the action log and all snapshots.

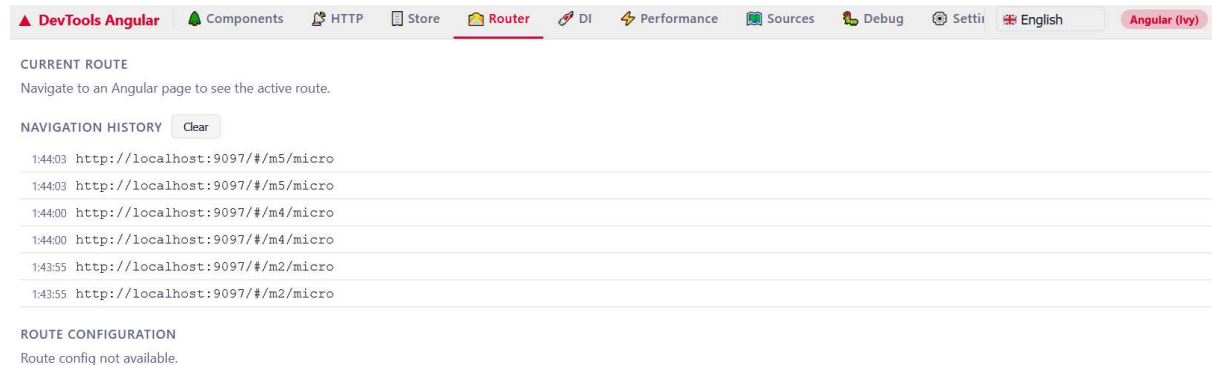
7.2 State Tree (Right Pane)

Displays the complete store state as an interactive JSON tree:

- Click ☐ / ☐ to expand or collapse any node.
- Use the Search keys field to filter by key name or value.
- Click **Current State** to jump back to the latest snapshot after time-travelling.
- Click **Export JSON** to copy the displayed state to the clipboard.

8. Router Tab

The Router tab provides real-time visibility into Angular's Router state: the active route, navigation history, and the full route configuration tree.



8.1 Current Route

Field	Description
URL	The complete URL currently active in the browser.
Path	The normalised route path segment.
Component	The component class associated with the active route.
Params	Route parameters (e.g. :id, :slug).
Query Params	Query string parameters (e.g. ?search=...&page=...).
Data	Static data defined in the route configuration.

8.2 Navigation History

A chronological list of all router navigation events:

- Red rows indicate a `NavigationError`.
- Amber rows indicate a `NavigationCancel` (e.g. rejected by a route guard).
- Click **Clear** to remove the history log.

8.3 Route Configuration Tree

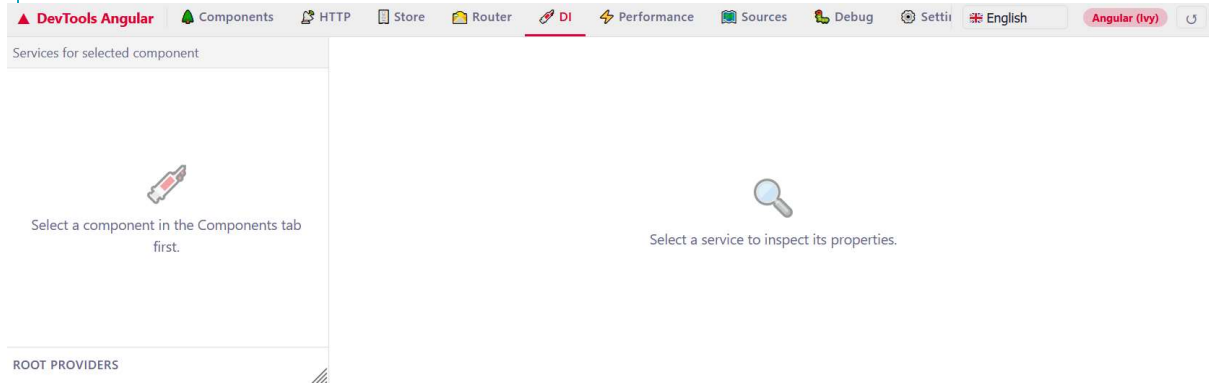
A collapsible tree of the full `Routes` array registered in the application:

- Lazy-loaded routes display a lazy badge.
- Routes with `canActivate` guards display a guard badge.
- The currently active route branch is highlighted in red.

9. DI Tab (Dependency Injection)

The DI tab shows the services injected into the component currently selected in the **Components** tab.

You must select a component in the Components tab first. The DI tab shows "Select a component in the Components tab first." until a selection is made.



9.1 Service List (Left Pane)

Column	Description
Name	TypeScript class name of the service.
Scope	root, module, or component — the injector level at which the service is provided.
⚠ Circular	Warning badge if a circular injection dependency is detected.

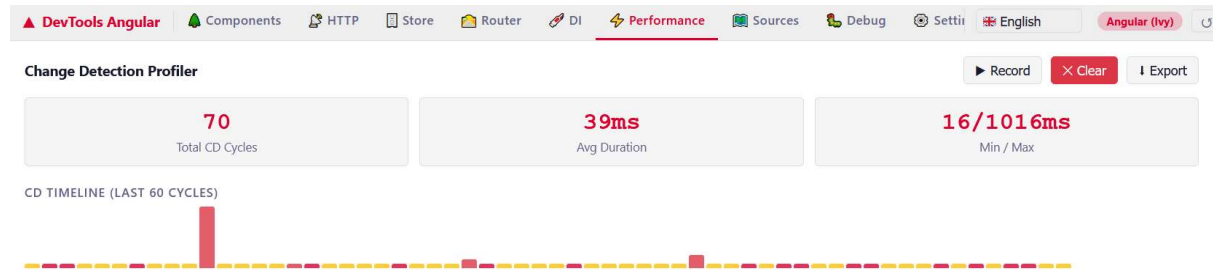
The **Root Providers** section at the bottom lists all providers registered at the root injector level.

9.2 Service Detail (Right Pane)

Click any service to see its public properties with their current runtime values.

10. Performance Tab

The Performance tab records Angular change detection (CD) cycles and helps identify components that trigger excessive re-renders.



10.1 Controls

Button	Action
☐ Record	Start instrumenting change detection cycles on the inspected page.
× Clear	Discard all recorded data.
☐ Export	Copy all recorded data as JSON to the clipboard.

10.2 Metrics Cards

Metric	Description
Total CD Cycles	Number of change detection cycles recorded since profiling started.
Avg Duration	Mean duration of a single cycle in milliseconds.
Min / Max	Fastest and slowest cycle durations recorded.

10.3 CD Timeline

A bar chart showing the duration of the last 60 change detection cycles:

Bar colour	Meaning
Normal	Duration within expected range.
Amber (slow)	Cycle took longer than 16 ms (more than one frame at 60 fps).
Red (critical)	Cycle took longer than 50 ms.

Hover over a bar to see the exact duration, trigger source, and timestamp.

10.4 Most Checked Components

A table of the 10 components checked most frequently by Angular's change detector. Components using the **Default** strategy that appear near the top of this list are

strong candidates for migration to **OnPush**, which can significantly reduce unnecessary re-renders.

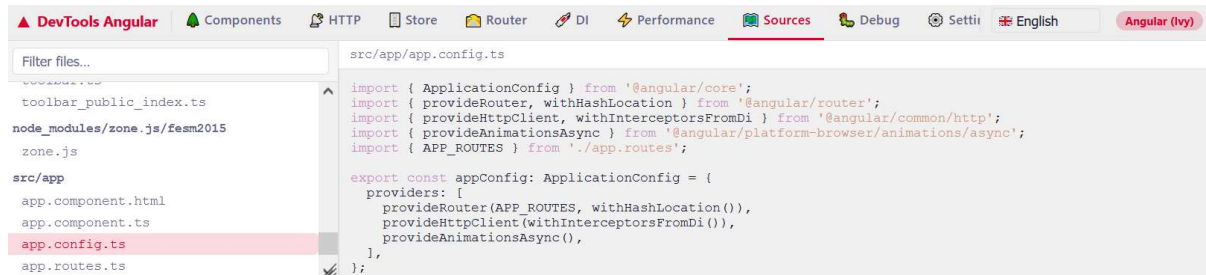
10.5 Subscription Leaks

If the extension detects Observable subscriptions that were not unsubscribed when a component was destroyed, an alert section lists the component name and the number of leaked subscriptions.

11. Sources Tab

The Sources tab displays the original TypeScript source files of the Angular application, loaded from the source map (.js.map) files that the development server serves alongside the compiled bundles.

Source maps are typically served in development mode (ng serve) and are rarely available on production deployments. If no source maps are found, the panel shows "No source maps detected yet."



11.1 File Browser (Left Pane)

Lists all .ts source files discovered from the available source maps, grouped by directory.

- Click any file name to display its contents in the right pane.
- Use the Filter files... field to filter by file name or directory path.

11.2 Source Viewer (Right Pane)

Displays the selected file with syntax highlighting for:

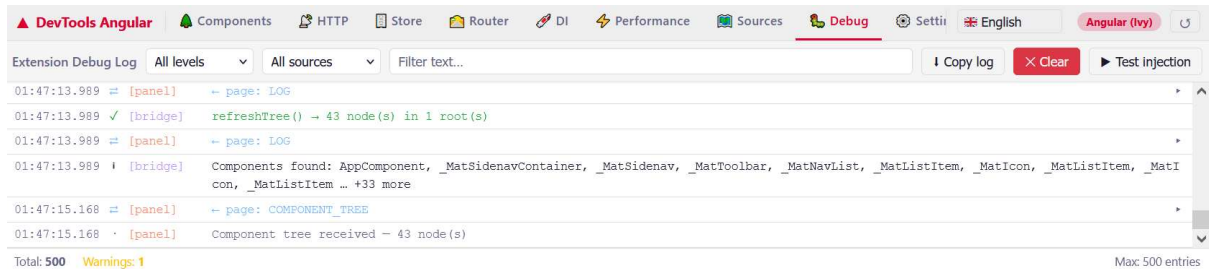
- Keywords (import, export, class, const, function, etc.)
- TypeScript decorators (@Component, @Injectable, etc.)
- String literals
- Numeric literals
- Single-line comments

11.3 Jump to Source

From the **Components** tab, when source maps are available, you can click **Jump to source** on a selected component. This automatically switches to the Sources tab and scrolls to the exact line where the component class is defined.

12. Debug Tab

The Debug tab shows an internal log of all messages exchanged between the extension's components (bridge, content script, panel, background service worker). It is intended for troubleshooting the extension itself.

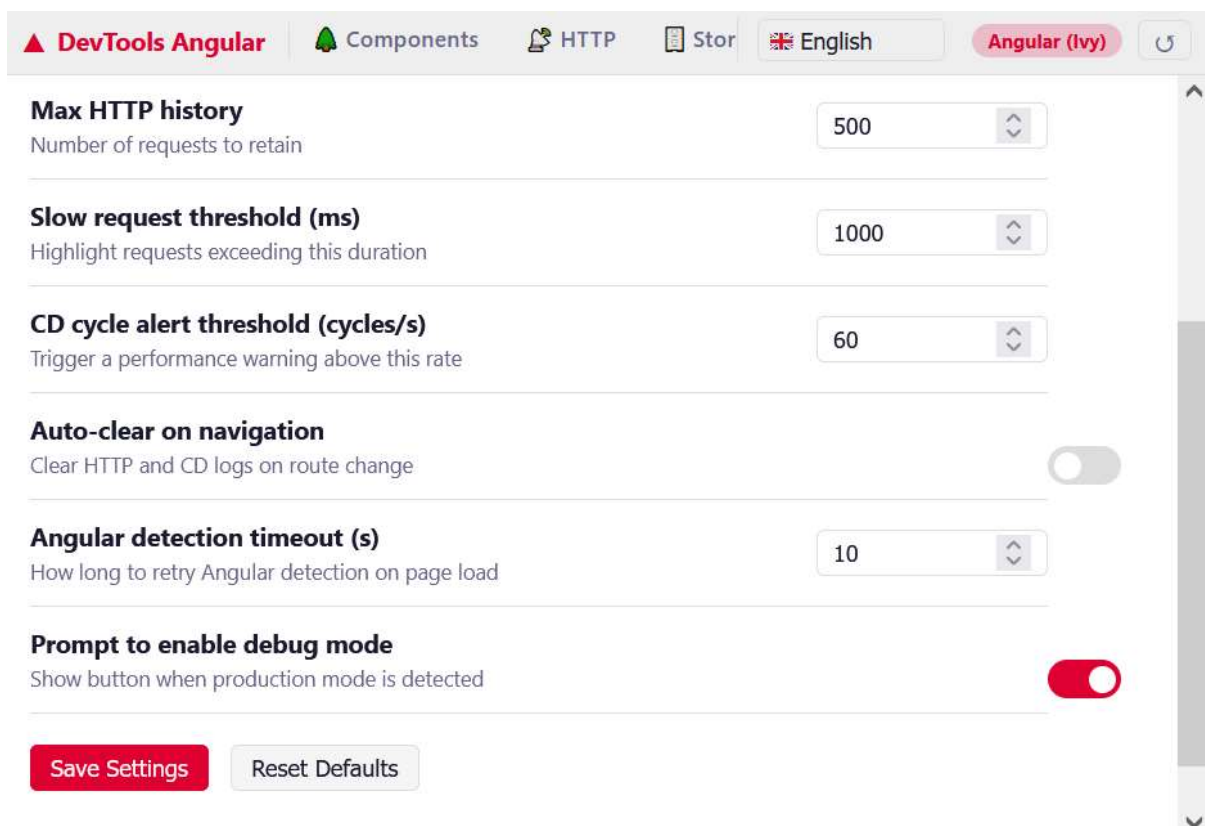
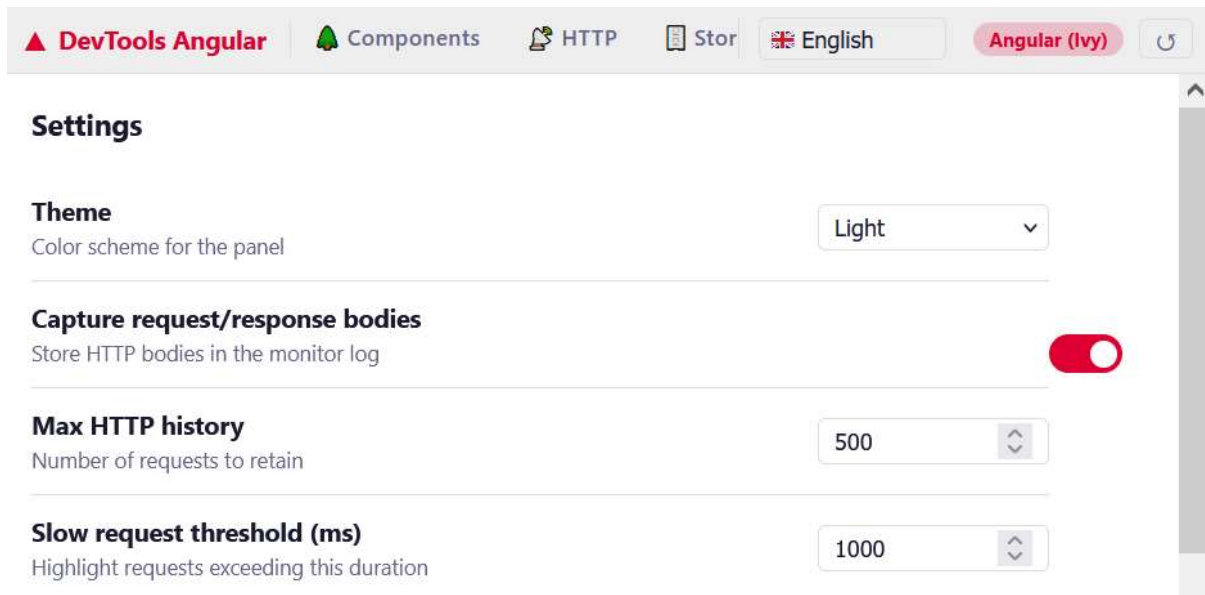


Log level	Colour / meaning
[INFO]	Informational message — normal operation.
[SUCCESS]	An operation completed successfully.
[WARN]	Something unexpected but non-fatal occurred.
[ERROR]	An error occurred that may affect functionality.
[BRIDGE]	A message was sent or received across the bridge between page and panel.
[DEBUG]	Verbose internal detail, generally not needed for day-to-day use.

Use the log to diagnose issues such as Angular not being detected, source maps not loading, or HTTP requests missing from the monitor.

13. Settings

The Settings tab lets you customise the extension's behaviour. Changes take effect immediately for theme-related settings and are saved when you click **Save Settings**.



Setting	Options / Range	Default
Theme	Dark / Light / System	Dark
Capture request/response bodies	On / Off	On
Max HTTP history	1 – 10 000 entries	500
Slow request threshold	Milliseconds (number)	1000 ms
CD cycle alert threshold	Cycles/second (number)	60
Auto-clear on navigation	On / Off	Off
Angular detection timeout	Seconds (number)	10 s
Prompt to enable debug mode	On / Off	On

Click **Save Settings** to persist changes to browser storage. Settings survive browser restarts and extension updates.

Click **Reset Defaults** to restore all values to their documented defaults. This also saves immediately.

13.1 Theme

- **Dark** — matches the Firefox DevTools dark theme (default).
- **Light** — inverts the panel to a white background.
- **System** — follows the operating system's color scheme preference.

13.2 HTTP Monitor Settings

- **Capture request/response bodies** — when disabled, request and response body content is not stored in memory. Only headers and timing are captured. Useful for pages with high request volume or large payloads.
- **Max HTTP history** — the number of requests retained in memory. Older entries are discarded when the limit is reached. Lower this value if the extension is consuming too much memory on pages with heavy HTTP traffic.
- **Slow request threshold** — requests taking longer than this value (in milliseconds) receive a red left border in the request list.

13.3 Performance Settings

- **CD cycle alert threshold** — the number of change detection cycles per second above which a warning is shown in the Performance tab.
- **Auto-clear on navigation** — when enabled, the HTTP log and CD profiling data are cleared automatically each time the Angular Router navigates to a new route.

14. Known Limitations

Limitation	Details
Production builds	Angular production builds remove the ng.* debug APIs. The component tree and DI inspector are unavailable. Click "Enable Debug Tools" to partially restore them. HTTP, Router, and Sources tabs remain functional.
Source maps in production	Most production deployments do not serve .map files. The Sources tab only works when source maps are actively served.
Zone.js-less apps (Angular 18+)	Applications using Angular 18+ without Zone.js may not trigger the change detection hooks the Performance profiler depends on.
Minified/obfuscated names	When a bundle is minified without source maps, component and service names appear as single-character identifiers (e.g. t, n, e).
Cross-origin iframes	Angular components inside a cross-origin <iframe> cannot be inspected due to browser security restrictions.
Requests before panel open	The HTTP interceptor is installed when the page loads. Requests that complete before the extension activates on a tab are not captured.
WebSocket traffic	WebSocket connections are not captured by the HTTP monitor.

15. Troubleshooting

The DevTools Angular tab does not appear in DevTools

- Verify the extension is installed and enabled (about:addons in Firefox, chrome://extensions in Chrome).
- Close and reopen DevTools (press **F12** twice).
- In Firefox, check about:debugging — the extension should appear in the list without errors.

The badge shows "No Angular" on an Angular application

- The app may still be bootstrapping. Reload the page with DevTools already open.
- For lazy-loaded SPAs, wait for the application to complete initialisation before the extension can detect Angular.
- Check the browser console for JavaScript errors that may have prevented the Angular bootstrap.
- Open the Debug tab to see the internal message log — it may show why detection failed.

The component tree is empty in production mode

- Click **Enable Debug Tools** in the amber production-mode banner. This invokes `enableDebugTools()` on the page context, which re-enables the `ng.*` APIs.
- If the production bundle is too aggressively optimised, this may not work. In that case, use the HTTP and Router tabs, which do not depend on `ng.*`.

HTTP requests are not appearing in the monitor

- Ensure the page was loaded after the extension was installed and active on this tab. The HTTP interceptor is injected at page load time.
- Some libraries that wrap `fetch()` or `XMLHttpRequest` before the extension injects may not be captured. Reload the page with DevTools open to ensure the interceptor runs first.

Source maps not detected

- Verify the development server is configured to serve source maps (ng serve does this by default).
- Check the Debug tab for `SOURCE_MAPS_FOUND` messages. If count is 0, the server is not serving `.map` files.
- Try reloading the page — source maps are discovered when the page initially loads its scripts.

The extension slows down after long sessions

- Reduce **Max HTTP history** in Settings to limit memory consumption.
- Disable **Capture request/response bodies** on pages with large HTTP payloads.
- Clear the HTTP log and CD profiling data periodically using the × buttons in each tab.

16. Privacy & Data

DevTools Companion for Angular operates entirely within your browser. It does not transmit any data to external servers.

Data type	Where it is stored
HTTP request/response data	In the DevTools panel's JavaScript memory only. Cleared when the panel is closed or the log is explicitly cleared.
Component state snapshots	In the DevTools panel's JavaScript memory only.
Store action log	In the DevTools panel's JavaScript memory only.
Extension settings (theme, thresholds, etc.)	In <code>browser.storage.local</code> — persisted on your device only.

No analytics, telemetry, or crash reporting is collected. You can verify this by monitoring the Network tab in Firefox's native DevTools while using the extension.

Appendix — Keyboard Shortcuts

Action	Shortcut
Open / close DevTools	F12 or Ctrl+Shift+I
Switch DevTools tabs	Click the tab label in the DevTools tab bar
Refresh component tree	Click  in the Components toolbar
Activate element picker	Click  in the Components toolbar
Accept edited @Input() value	Tab or click outside the field
Clear HTTP log	Click  in the HTTP toolbar
Start/stop CD profiling	Click  Record /  in the Performance tab